- Guía: Desarrollo de Software con IA Agéntica
  - Basado en el proyecto Pharmadix Times (Cibertec 2026)
  - 1. ¿Qué es la IA Agéntica para Desarrollo de Software?
    - Diferencia vs. usar ChatGPT/Copilot convencional
  - 2. Herramientas Utilizadas
    - 2.1 Editor + Agente Principal
    - 2.2 MCPs (Model Context Protocol Servers)
      - Cómo instalar MCPs en Cursor
    - 2.3 Skills (Habilidades Especializadas)
  - 3. Metodología: Desarrollo sin Alucinaciones
    - El Principio Fundamental
    - 3.1 Flujo de Trabajo Correcto
    - 3.2 Patrón para Documentar (sin inventar)
    - 3.3 Patrón para Errores de Compilación
  - 4. Uso de Google Stitch MCP para Mockups Android
    - 4.1 Cómo generar pantallas
    - 4.2 Cómo exportar los mockups al informe
    - 4.3 Cómo usar los mockups para implementar la app
  - 5. Uso de mcp-mermaid para Diagramas en Documentos
    - El problema
    - La solución con el MCP
- Usar el MCP para generar la imagen PNG:
- Resultado: imagen PNG que SIEMPRE se ve igual en PDF/Word/GitHub
  - 8. Anti-patrones: Lo que NO hacer
    - ❌ Decirle al agente "crea un DAO" sin leer el proyecto primero
    - ❌ Copiar código de ejemplo genérico de ChatGPT al proyecto
    - ❌ Confiar en que los diagramas Mermaid van a renderizar en todos lados
    - ❌ Asumir que el backend tiene usuarios creados
  - 9. Recurso: Links del Proyecto

Guía: Desarrollo de Software con IA Agéntica

Basado en el proyecto Pharmadix Times (Cibertec 2026)

> Esta guía documenta la metodología real usada en el desarrollo del proyecto **Pharmadix Times**: una app Android nativa + Backend Node.js + Portal Web React, construidos con ayuda de IA agéntica en Cursor/Antigravity, sin alucinaciones.

---

1. ¿Qué es la IA Agéntica para Desarrollo de Software?

La **IA agéntica** no es un chatbot que "sugiere código" — es un agente que:

1. **Lee archivos reales** del proyecto antes de escribir código
2. **Ejecuta comandos** y verifica los resultados
3. **Toma decisiones** basándose en el output real (no en suposiciones)
4. **Itera** hasta que el código funciona, equal que un desarrollador humano

Diferencia vs. usar ChatGPT/Copilot convencional

IA Convencional	IA Agéntica (Antigravity)
Genera código desde cero (puede alucinar APIs)	Lee el código existente antes de escribir
No sabe si el código compila	Ejecuta `./gradlew assembleDebug` y lee el error
Genera versiones de librerías incorrectas	Lee `build.gradle.kts` y usa las versiones reales
Asume cómo funciona el backend	Lee `routes/auth.ts` antes de configurar Retrofit
No puede verificar que el seed corrió	Ejecuta `docker exec` y verifica el output

---

2. Herramientas Utilizadas

2.1 Editor + Agente Principal

Cursor con el agente **Antigravity**:

- Acceso al filesystem completo del proyecto
- Ejecución de comandos en terminal (PowerShell/bash)
- Capacidad de editar múltiples archivos en una sola operación
- Modo planning (planifica antes de ejecutar) y modo ejecución

2.2 MCPs (Model Context Protocol Servers)

Los MCPs son extensiones que dan capacidades especiales al agente:

MCP	Para qué se usó	Ejemplo real
StitchMCP (Google)	Diseñar mockups de pantallas Android desde texto	`generate_screen_from_text("Pharmadix login screen with blue theme")`
mcp-mermaid	Generar diagramas como imágenes PNG	`generate_mermaid_diagram(flowchart TD...)` con `outputType: "file"`
pencil	Diseño de componentes UI en .pen files	Crear y editar diseños vectoriales
notebooklm	Consultar documentación en NotebookLM	Preguntar sobre tecnologías específicas

Cómo instalar MCPs en Cursor

Los MCPs se configuran en `.gemini/antigravity/mcp_config.json` o en la configuración de Cursor. Ejemplo:

```
{
  "mcpServers": {
    "StitchMCP": {
      "command": "npx",
      "args": ["-y", "@google/stitch-mcp"]
    },
    "mcp-mermaid": {
      "command": "npx",
      "args": ["-y", "mcp-mermaid"]
    }
  }
}
```

2.3 Skills (Habilidades Especializadas)

Las skills son archivos `SKILL.md` que le dan al agente instrucciones específicas para tareas complejas. Se ubican en `.cursor/skills/` del proyecto o globalmente en `~/.cursor/skills/`.

Skill usada en este proyecto:

```
.cursor/skills/extract-requirements-db-design/
├── SKILL.md         # Instrucciones para extraer requisitos y diseñar BD desde
documentos
└── examples/        # Ejemplos de uso
```

Cómo activar una skill: Simplemente menciona la tarea en lenguaje natural. El agente detecta cuándo usar una skill basándose en la descripción de la tarea.

Cómo guardar una skill globalmente (disponible en todos los proyectos):

```
$global = "$env:USERPROFILE\.cursor\skills\extract-requirements-db-design"
New-Item -ItemType Directory -Force -Path $global
Copy-Item -Recurse ".cursor\skills\extract-requirements-db-design\*" $global
```

---

3. Metodología: Desarrollo sin Alucinaciones

El Principio Fundamental

> **"El agente nunca asume — siempre verifica primero"**

Antes de escribir cualquier código, el agente:

1. Lee los archivos relevantes (entities, DAOs, configuración)
2. Verifica las versiones de las dependencias reales
3. Ejecuta el comando y lee el output de error antes de "arreglar" algo

3.1 Flujo de Trabajo Correcto

```
[USUARIO] "Agrega Room a la app Android"
     │
     ▼
[AGENTE] Lee build.gradle.kts actual
     │   → Constata que AGP es 9.0.1
     │   → Constata que ya existe KSP
     │
     ▼
[AGENTE] Agrega Room 2.7.0 (compatible con KSP 2.0.21)
     │   → NO inventa una versión random
     │
     ▼
[USUARIO] Ejecuta Sync → Error: "mipmap-anydpi requires SDK 26"
     │
     ▼
[AGENTE] Lee la estructura de res/ real
     │   → Detecta carpeta mipmap-anydpi/
     │   → Mueve los XMLs a mipmap-anydpi-v26/
     │
     ▼
[AGENTE] Verifica con find_by_name que los archivos se movieron
```

3.2 Patrón para Documentar (sin inventar)

Cuando el usuario pidió documentar el código, el agente:

1. **Primero leyó todos los archivos reales:**
   - `EmpleadoDao.kt` → copió el código exacto al informe
   - `RetrofitClient.kt` → copió la implementación real
   - `item_registro_operario.xml` → copió el XML completo

2. **NO generó código de ejemplo inventado**, sino el código real del proyecto

3. **Verificó que existían los archivos** antes de referenciarlos

```
# Ejemplo de lo que el agente hizo:
find_by_name("*Dao*") → encontró EmpleadoDao.kt, HojaTiempoDao.kt,
RegistroTiempoDao.kt
view_file(EmpleadoDao.kt) → leyó el contenido exacto
# LUEGO lo insertó en el informe
```

3.3 Patrón para Errores de Compilación

Cuando ocurre un error, el flujo correcto es:

```
[ERROR] "failed to connect to /10.0.2.2 (port 3000)"
     │
     ▼
[AGENTE lee] → RetrofitClient.kt: BASE_URL = "http://10.0.2.2:3000/"
              → docker-compose.yml: ports: "3000:3000"
              → .env: PORT=3000
     │
     ▼
[CONCLUSIÓN] La URL es correcta. El problema es que el backend no tiene datos.
[ACCIÓN] Ejecutar el seed: docker exec pharmadix-backend npx ts-node prisma/seed.ts
[VERIFICACIÓN] Invoke-WebRequest http://localhost:3000/auth/login → responde 200
```

---

4. Uso de Google Stitch MCP para Mockups Android

4.1 Cómo generar pantallas

```
# En el chat con el agente:
"Genera en Stitch una pantalla de Login para app Android con tema azul #1A237E"

# El agente llama internamente:
mcp_StitchMCP_generate_screen_from_text({
  projectId: "17039988028003338139",
  prompt: "Pharmadix Times login screen, dark blue #1A237E theme...",
```

```
  deviceType: "MOBILE"
})
```

4.2 Cómo exportar los mockups al informe

```
# 1. Listar screens disponibles:
mcp_StitchMCP_list_screens({ projectId: "..." })

# 2. Obtener screenshot de cada screen:
mcp_StitchMCP_get_screen({
  name: "projects/.../screens/...",
  projectId: "...",
  screenId: "..."
})

# 3. El agente descarga el PNG y lo referencia en el MD:
![Mockup Login](mockup_login.png)
```

4.3 Cómo usar los mockups para implementar la app

Los screens de Stitch tienen código HTML exportable. El flujo fue:

1. Diseñar el mockup en Stitch (primero el boceto visual)
2. Usar el mockup como guía para el XML de Android:
   - Colores del mockup → `colors.xml`
   - Componentes del mockup → `activity_login.xml`, `fragment_registro_operarios.xml`
   - Layout del mockup → `ConstraintLayout` con ids correspondientes

---

5. Uso de mcp-mermaid para Diagramas en Documentos

El problema

Los editores de Markdown (GitHub, VS Code 8.8.0) no soportan todas las versiones de Mermaid. Los bloques ```mermaid` pueden dar error.

La solución con el MCP

```
# En lugar de poner código mermaid que puede fallar:
```mermaid
flowchart TD  ← NO soportado en 8.8.0
```

Usar el MCP para generar la imagen PNG:

mcp_mcp-mermaid_generate_mermaid_diagram({ mermaid: "graph TD\n A --> B", ← sintaxis compatible 8.8 outputType: "file" ← guarda PNG en disco })

Resultado: imagen PNG que SIEMPRE se ve igual en PDF/Word/GitHub

Diagrama

```
### Sintaxis compatible Mermaid 8.8.0

| ❌ No usar | ✅ Usar en su lugar |
|---|---|
| `flowchart TD` | `graph TD` |
| `stateDiagram-v2` | `stateDiagram` |
| `subgraph id [label]` | `subgraph id` |
| Emojis en labels | Texto plano |
| `\n` en labels | ` - ` (guión) |
| `<br/>` en nodos | Sin saltos de línea |

---

## 6. Modo Planning: Planificar Antes de Ejecutar

Para tareas grandes, el agente usa "Planning Mode" que:
1. **Lee todo** lo necesario primero
2. **Crea un plan** en `implementation_plan.md`
3. **Pide aprobación** antes de ejecutar
4. **Ejecuta por fases** verificando cada una
```
```

```
### Ejemplo real de este proyecto
```

USUARIO: "Agrega las secciones de código faltantes al informe académico"

AGENTE (Planning):

1. Lee EmpleadoDao.kt, RegistroTiempoDao.kt, HojaTiempoDao.kt
2. Lee ApiService.kt, RetrofitClient.kt, ApiModels.kt
3. Lee item_registro_operario.xml
4. Lee el informe para encontrar las líneas exactas de inserción
5. Crea implementation_plan.md con los 3 bloques a agregar
6. Pide aprobación ──
   USUARIO: "Aprueba"
7. Ejecuta Bloque 1: Room entities + DAOs → sección 6.2 del informe
8. Ejecuta Bloque 2: REST services → sección 6.4 del informe
9. Ejecuta Bloque 3: XML RecyclerView → sección 7.2 del informe

```
---

## 7. Stack Recomendado para Proyectos Similares

### Para un proyecto académico Android + Backend
```

Cursor (editor) + Antigravity (agente) ├── MCPs: │ ├── Google Stitch → mockups móviles │ └── mcp-mermaid → diagramas como imágenes ├── Skills: │ └── extract-requirements-db-design → análisis de documentos └── Flujo de trabajo: 1. ANALIZA → lee el doc de requisitos con la skill 2. DISEÑA → crea mockups con Stitch MCP 3. PLANEA → modo planning con implementation_plan.md 4. DESARROLLA → escritura de código real (sin alucinaciones) 5. VERIFICA → ejecuta comandos y lee el output 6. DOCUMENTA → copia código real al informe (no inventa) 7. EXPORTA → diagramas como PNG con mcp-mermaid

```
### Comandos útiles para verificar el backend

```powershell
# Verificar que Docker está corriendo
docker-compose ps
```
```

```
# Ver logs del backend en tiempo real
docker logs -f pharmadix-backend

# Probar endpoint de login
Invoke-WebRequest -Uri "http://localhost:3000/auth/login" `
  -Method POST `
  -ContentType "application/json" `
  -Body '{"usuario":"tomador1","password":"password123"}' `
  -UseBasicParsing | Select-Object -ExpandProperty Content

# Correr el seed si los usuarios no existen
docker exec pharmadix-backend npx ts-node prisma/seed.ts
```

---

8. Anti-patrones: Lo que NO hacer

❌ Decirle al agente "crea un DAO" sin leer el proyecto primero

El agente puede inventar una estructura que no coincida con el schema real.

✅ **Correcto:** "Lee los archivos existentes en `data/local/` y agrega un DAO para la entidad X"

❌ Copiar código de ejemplo genérico de ChatGPT al proyecto

El código puede usar versiones incompatibles de librerías.

✅ **Correcto:** El agente lee `build.gradle.kts` y usa exactamente las versiones declaradas.

❌ Confiar en que los diagramas Mermaid van a renderizar en todos lados

Cada visor usa una versión diferente.

✅ **Correcto:** Usar mcp-mermaid para exportar PNGs → funcionan en TODOS los contextos.

❌ Asumir que el backend tiene usuarios creados

El seed es una operación separada que hay que ejecutar explícitamente.

✅ **Correcto:** `docker exec pharmadix-backend npx ts-node prisma/seed.ts`

---

9. Recurso: Links del Proyecto

- 🐙 **GitHub**: https://github.com/Josesaso45/Pharmadix_Form_Horas_Extras_Cibertec
- 🎨 **Stitch Project**: https://stitch.withgoogle.com (proyecto: Pharmadix Times – App Android MVP)
- 📋 **Informe Académico**: `Documentacion_Proyecto_Cibertec/Pharmadix_Times_Informe_Proyecto.md`
- 📝 **Cursor**: https://cursor.sh
- 🔧 **Mermaid Live**: https://mermaid.live (para probar sintaxis antes de usar el MCP)

Generado el 27 de Febrero, 2026 · Proyecto Pharmadix Times · Cibertec DAM I